Angular Advanced
Day 1 – short recap

Peter Kassenaar –
info@kassenaar.com

Yesterday:

- Some Angular CLI tips & tricks
- NG applications with multiple modules
  - Routing and Lazy loading modules
  - Custom Loading strategies
  - Child Routes – nested router outlets
- Content projection
  - <ng-content [select="…"]></ng-content>
  - different options
  - Mostly used when building reusable components

- Event Buses/ Service buses
  - @Input() & @Output() - direct parent-child communication
  - @Injectable() Service, exposing a `Subject()`
- Named Router Outlets / Aux Routes
  - Multiple `<router-outlets>`'s in one page
  - URL reflects state in address bar
  - Log, or subscribe to routing events

```
let routerConfig: ExtraOptions = {
  enableTracing: true
};

@NgModule({
  imports: [RouterModule.forRoot(routes, routerConfig)],
  exports: [RouterModule]
})
```

Routing events are actually Observables.

Which means we can subscribe! And do something like:

```
this.router.events
    .subscribe(event =>{
      console.log(' router event: ', event);
  })
```

(Don’t forget to inject router:

```
constructor(private router: Router) {  }
```

Or, in a more reactive way of programming:

```
this.router.events
    .filter(event => event instanceof NavigationEnd)
    .map(...)
    . ...
    .subscribe(event =>{
      console.log(' router event: ', event);
  })
```

Today

- Wrap up Named Router Outlets
- Angular 4.3+ : HttpClient
- State Management `@ngrx/store`
- Testing

Angular 4.3+: `HttpClientModule`

- In @ngModule: `imports : [HttpClientModule]`
- No more `.map(res => res.json()).`
  - Json is default!
  - Add params if you want access to raw Response Object
- New: Interceptors
- https://alligator.io/angular/httpclient-intro/ and
- https://alligator.io/angular/httpclient-interceptors/

- …is default in Angular 5
  - `HttpModule` will be removed in future versions
  - Update your Angular-CLI!
  - `HttpModule` (Angular 4) vs.
  - `HttpClientModule` (Angular 4.3+)

https://alligator.io/angular/httpclient-intro/

RxJS 5.5+: lettable operators en `.pipe()`

This repository Search Pull requests Issues Marketplace Explore

ReactiveX / rxjs

Watch 357 Star 9,548 Fork 905

Code Issues 225 Pull requests 68 Projects 3 Insights

Branch: master rxjs / doc / lettable-operators.md Find file Copy path

seniorquico docs: change "pipe" utility function module path from "utils" to "uti... 39c2e7c on Oct 25

6 contributors

237 lines (187 sloc) 9.12 KB Raw Blame History

Lettable Operators

Starting in version 5.5 we have shipped "lettable operators", which can be accessed in `rxjs/operators` (notice the pluralized "operators"). These are meant to be a better approach for pulling in just the operators you need than the "patch" operators found in `rxjs/add/operator/*`.

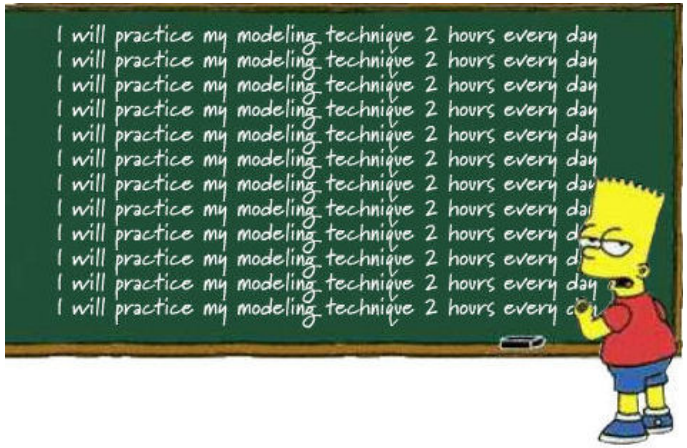
NOTE: Using `rxjs/operators` without making changes to your build process can result in larger bundles. See Known Issues section below.

Renamed Operators

Due to having operators available independent of an Observable, operator names cannot conflict with JavaScript keyword restrictions. Therefore the names of the lettable version of some operators have changed. These operators are:

https://github.com/ReactiveX/rxjs/blob/master/doc/lettable-operators.md

```
// new: rxjs 5.5 lettable operators with .pipe()
return
this.http.get<City[]>('assets/data/cities.json')
  .pipe(
    tap(res => console.log(res)),
    catchError(err => {
      console.log(err);
      return Observable.of([])
    })
  )
```

Workshop

- Use the example `../140-named-router-outlets`
- Replace old skool `HttpModule` with `HttpClientModule`:
  - Update `app.module.ts`
  - Update `city.service.ts`
- Read post on `HttpClientModule` by Alligator
  - Optional: create an `Interceptor` – Use it simply to log current Time and the requested page to the console.